CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G1
SYSTEMS INTEGRATION AND ARCHITECTURE 1

FUNCTIONAL REQUIREMENTS SPECIFICATION (FRS)

Project Title: Mini App - User Registration & Authentication

Prepared By: Josh Anton K. Nuevas

Date of Submission: 02/06/2026

Version: 2.0

Table of Contents

1. Introduction....3
   1.1. Purpose....3
   1.2. Scope....3
   1.3. Definitions, Acronyms, and Abbreviations....3
2. Overall Description....3
   2.1. System Perspective....3
   2.2. User Classes and Characteristics....3
   2.3. Operating Environment....3
   2.4. Assumptions and Dependencies....3
3. System Features and Functional Requirements....3
   3.1. Feature 1:....3
   3.2. Feature 2:....3
4. Non-Functional Requirements....3
5. System Models (Diagrams)....4
   5.1. ERD....4
   5.2. Use Case Diagram....4
   5.3. Activity Diagram....4
   5.4. Class Diagram....4
   5.5. Sequence Diagram....4
6. Appendices....4

1. Introduction

1.1. Purpose

This system serves as a secure access point that handles user authentication, credentials, and personal profile management within the application. It is designed for use by developers, project stakeholders, and system administrators.

1.2. Scope

The system will manage user registration, authentication, and basic profile management. It functions as the core security layer of the "Mini App," while excluding third-party payment services and advanced social media integrations at this stage.

1.3. Definitions, Acronyms, and Abbreviations

- **FRS (Functional Requirements Specification):** A document that outlines the required behaviors and functionalities of the system.
- **UI (User Interface):** The area where users interact directly with the application.
- **API (Application Programming Interface):** A set of protocols that enables communication between the React-based frontend and the Spring Boot backend.
- **ERD (Entity Relationship Diagram):** A diagram that illustrates the database structure and the relationships among data entities.
- **JWT (JSON Web Token):** A secure mechanism for exchanging information in JSON format, utilized for user authentication.
- **BCrypt:** A cryptographic hashing algorithm used to safely store user passwords in the database.
- **ReactJS:** A JavaScript library employed to develop the user interface and manage frontend logic.
- **Spring Boot:** A Java framework used to build RESTful APIs and implement backend business processes.
- **MySQL:** A relational database system used to store user credentials and profile information.

2. Overall Description

2.1. System Perspective

The authentication system functions as an independent module that connects with the main "Mini App" database to validate user identities before allowing access to secured features. It follows a three-tier client–server architecture, where the ReactJS frontend manages the user interface, the Spring Boot backend handles business logic through APIs, and the MySQL database ensures persistent data storage.

2.2. User Classes and Characteristics

- **Unregistered User:** Has access to the home page and can create an account.
- **Registered User:** Can sign in, view and manage their profile, and use the application's features.

2.3. Operating Environment

The system operates within modern web browsers on the client side, utilizes a Java-based Spring Boot REST framework for backend processing, and relies on MySQL as the primary data storage solution.

2.4. Assumptions and Dependencies

- Users are expected to have a stable internet connection to enable proper communication between the ReactJS frontend and the Spring Boot backend.
- The system relies on the backend REST API being correctly configured, continuously available, and accessible by the frontend application.
- The application assumes that the MySQL database is active, properly structured, and capable of securely storing user credentials.
- It is assumed that users access the system through a modern web browser that supports and can execute ReactJS applications.

3. System Features and Functional Requirements

3.1. Feature 1:

Description: Allows new users to create an account by submitting valid registration details.
Functional Requirements:
- The system shall allow guest users to register using a unique email address and a secure password.
- The system shall validate all required input fields to maintain data accuracy and integrity.
- The system shall encrypt user passwords before storing them in the MySQL database.
- The system shall store user information only after successful validation and completion of the registration process.

3.2. Feature 2:

Description: Allows registered users to log in, access secured system features, and log out safely.
Functional Requirements:
- The system shall authenticate users by verifying their login credentials against stored database records.
- The system shall generate a secure authentication token (such as JWT) upon successful login.
- The system shall restrict access to protected pages and dashboards for unauthenticated users.
- The system shall end the active session and invalidate the authentication token when a user logs out.

4. Non-Functional Requirements

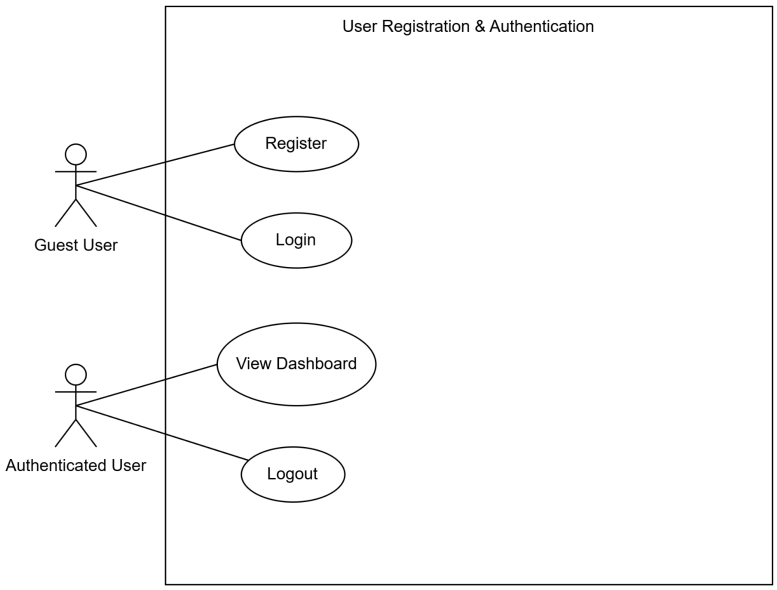
- **Security:** User passwords shall be securely hashed using strong encryption methods such as BCrypt before being stored in the database.
- **Performance:** The system shall handle registration and login requests within an acceptable response time to ensure a smooth user experience.
- **Usability:** The user interface shall be designed to be simple, intuitive, and easy to navigate for all types of users.
- **Reliability:** The system shall handle invalid inputs and authentication failures gracefully by providing clear and informative feedback.
- **Availability:** System functionality depends on stable internet connectivity and the proper configuration and availability of backend services.

5. System Models (Diagrams)

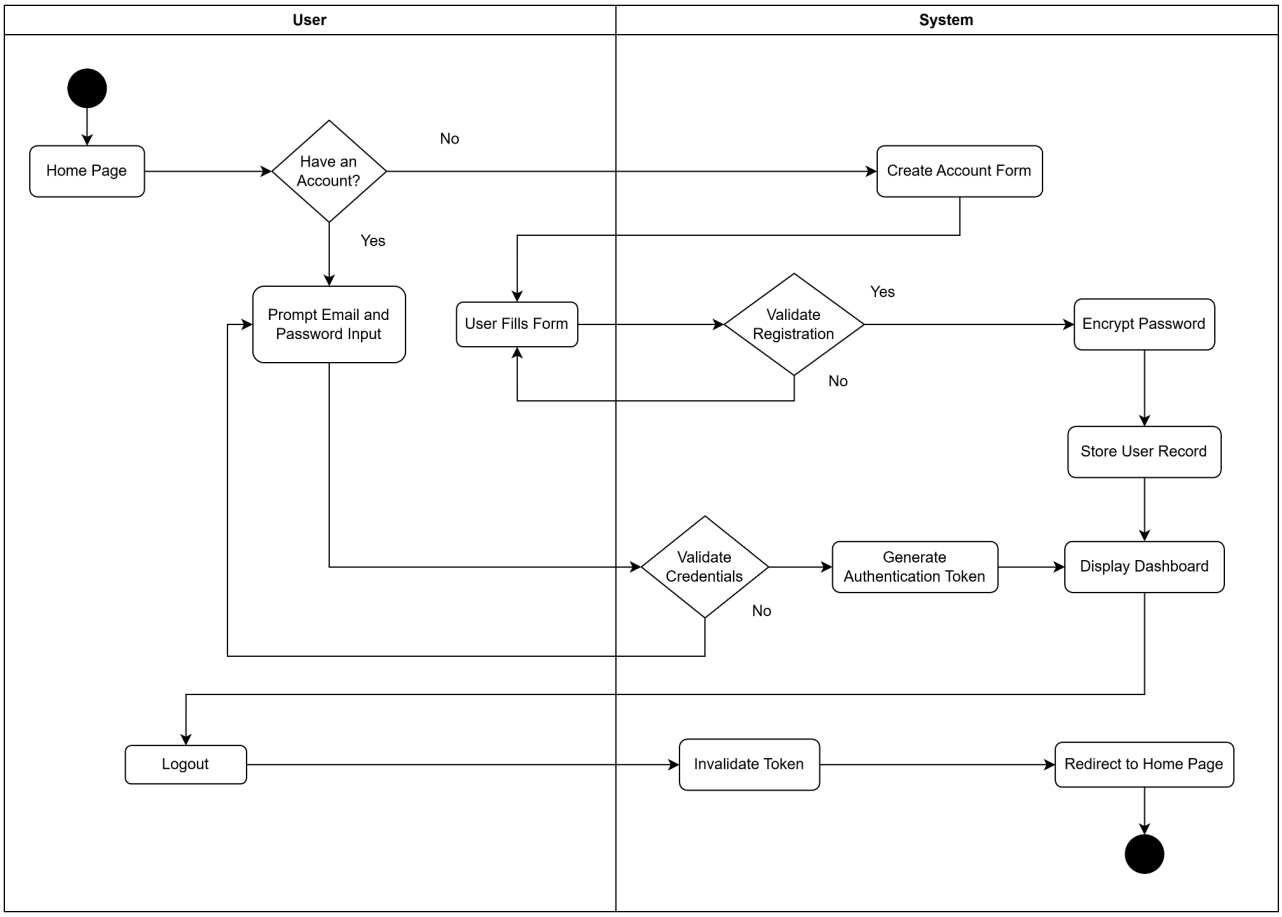
5.1. ERD

User	
PK	<u>user_id</u>
	name
	email
	password
	phone
	address

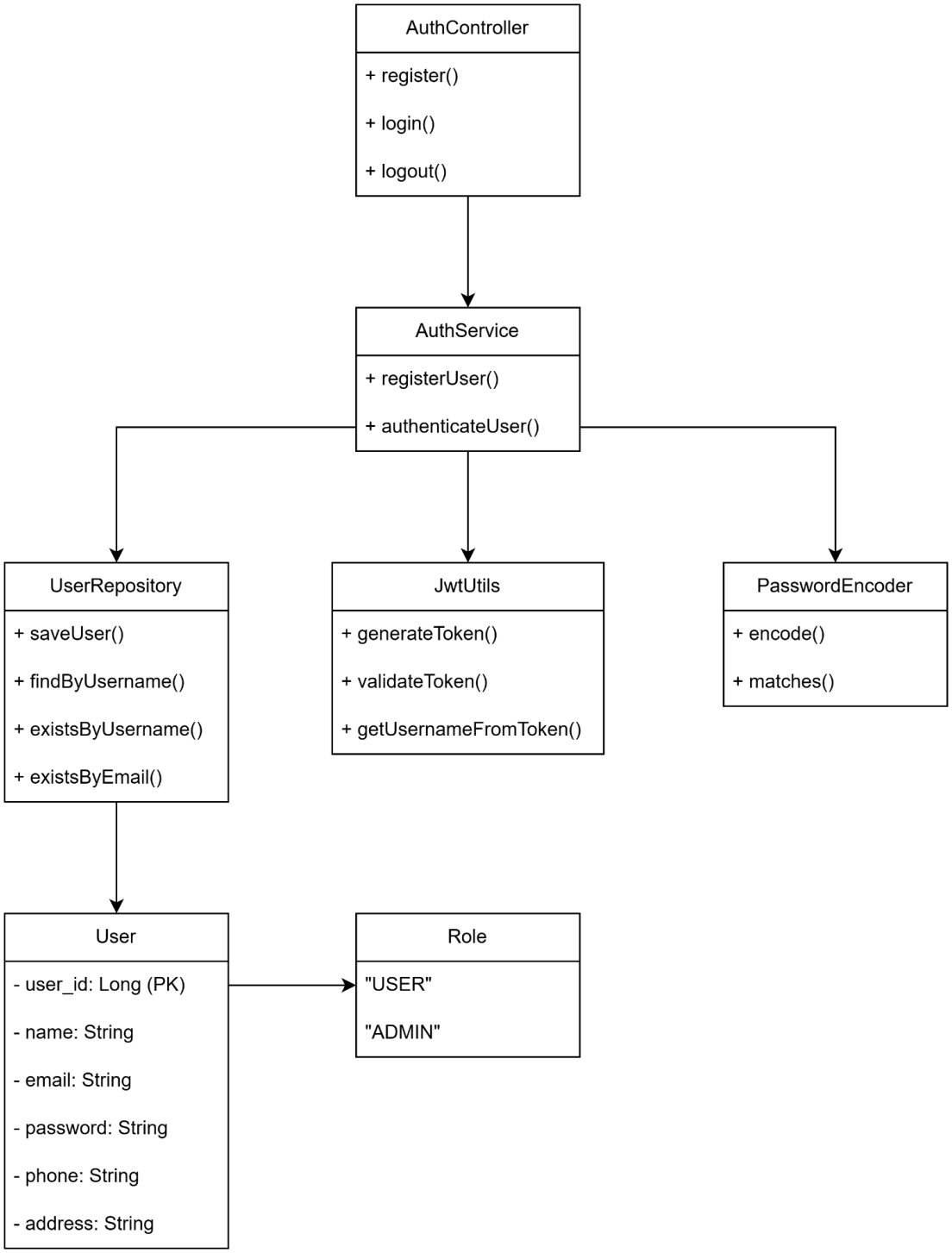
5.2. Use Case Diagram



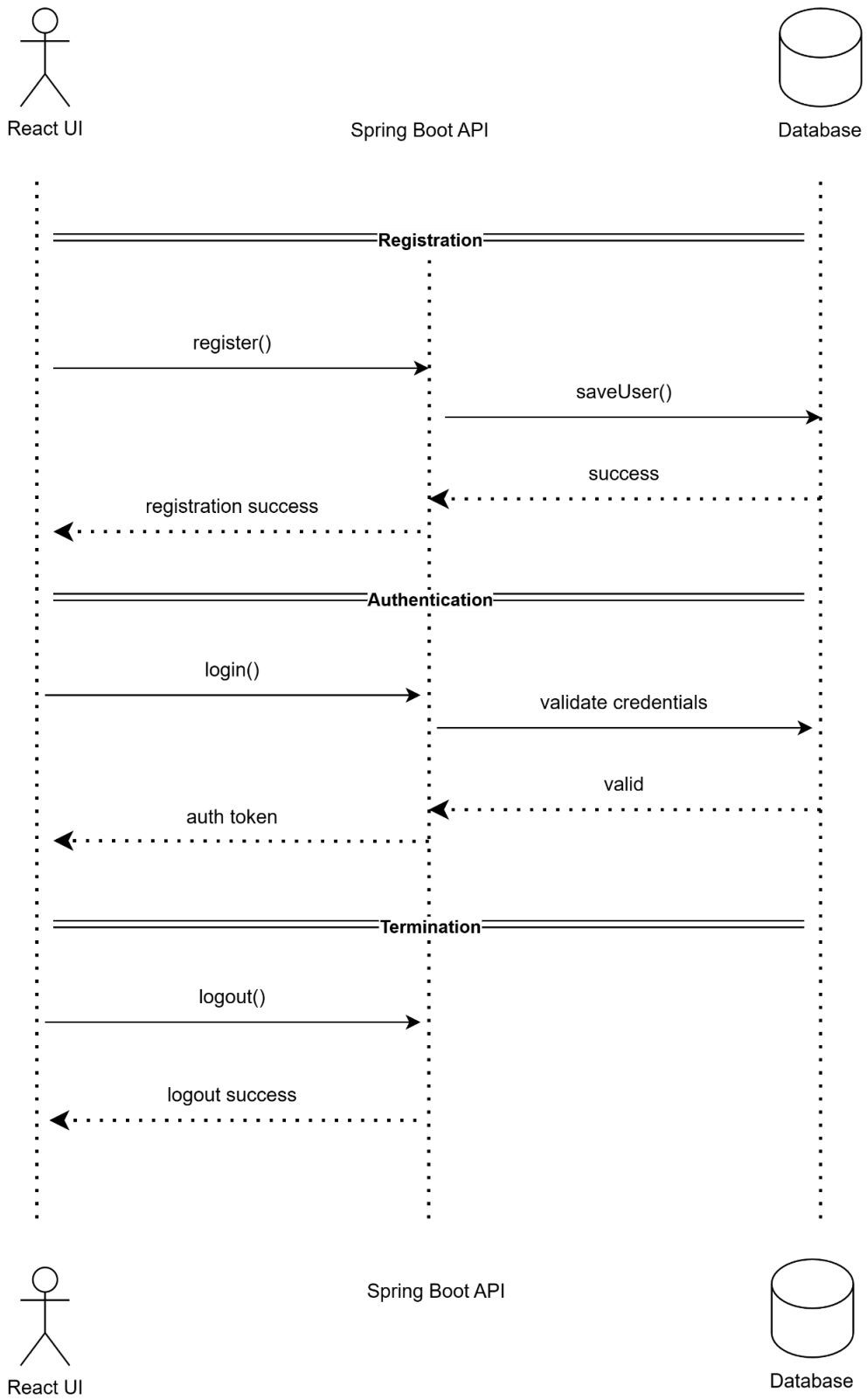
5.3. Activity Diagram



5.4. Class Diagram



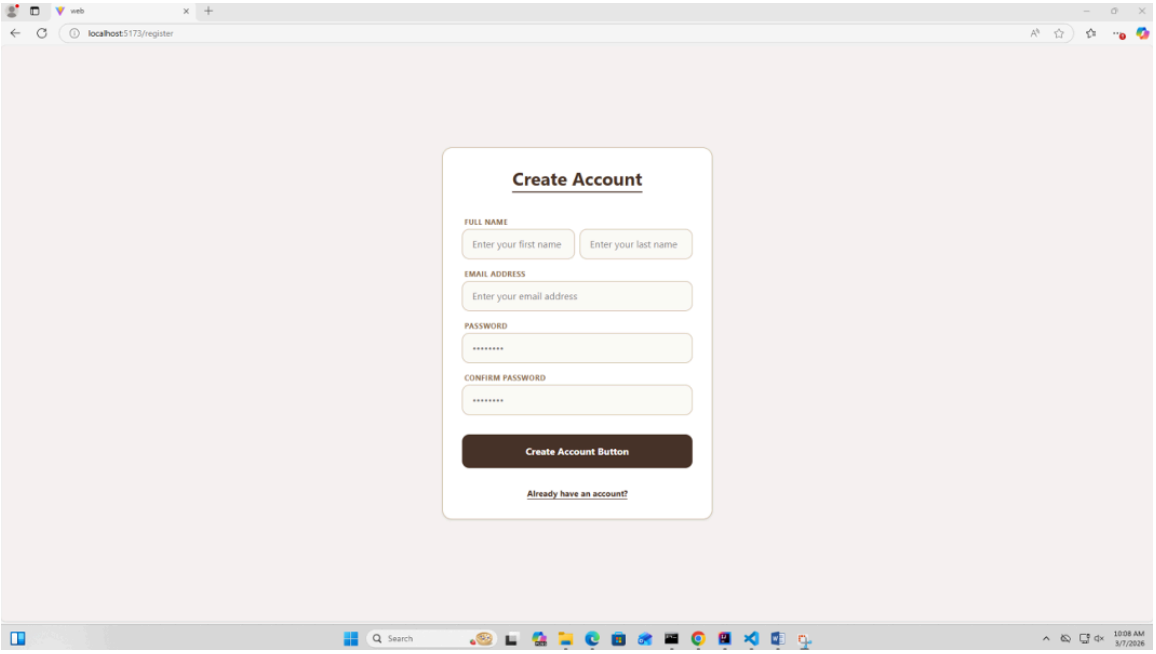
5.5. Sequence Diagram

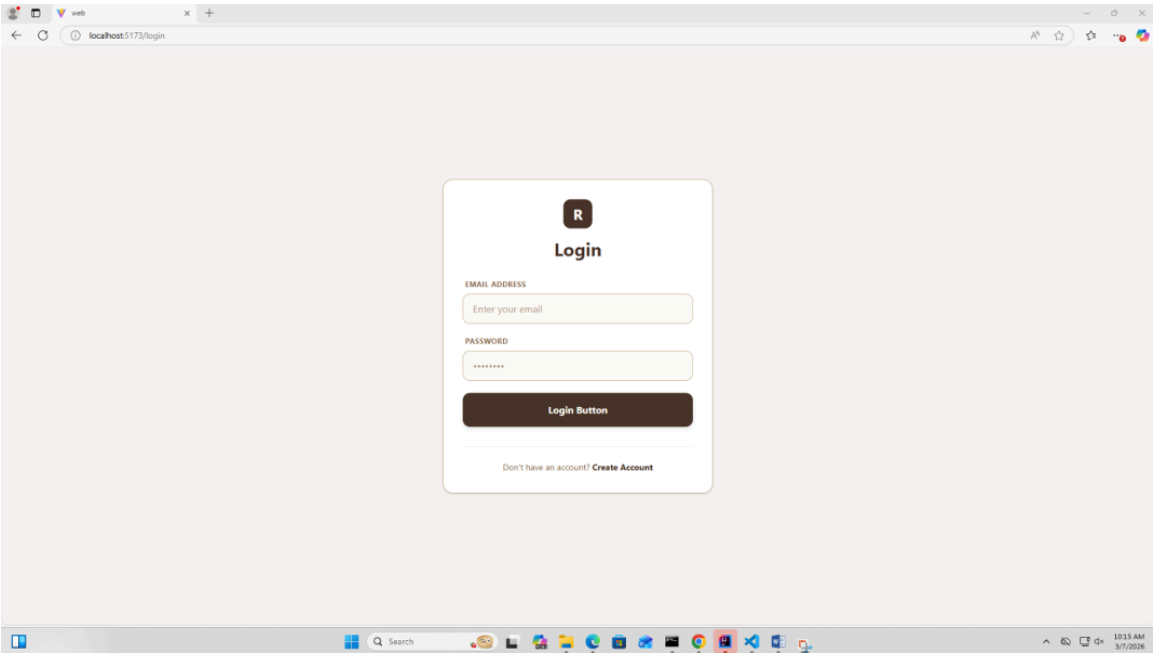


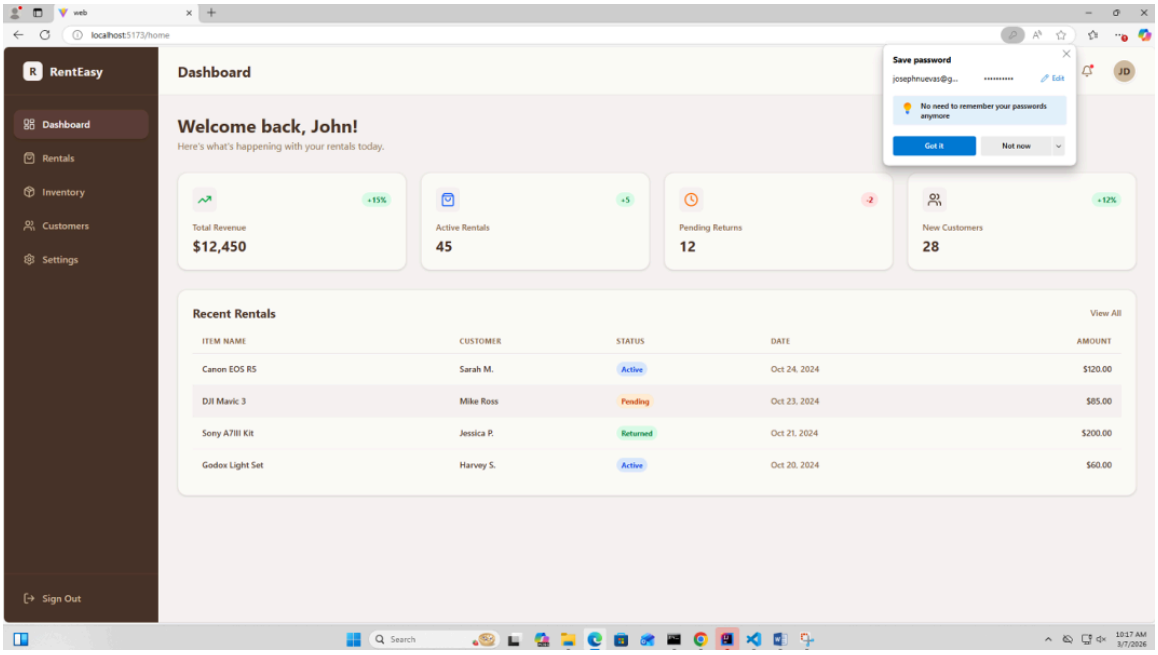
6. Appendices

- **Technological Stack:** The system is implemented using ReactJS for the user interface, Spring Boot for API development, and MySQL for database management.
- **Security Measures:** Passwords are protected through BCrypt hashing, while JSON Web Tokens (JWT) are used to manage secure user sessions.
- **Design Tools:** All system and database diagrams were created using draw.io / diagrams.net.
- **External Dependencies:** The system requires reliable internet connectivity and properly configured backend and database services to operate effectively.

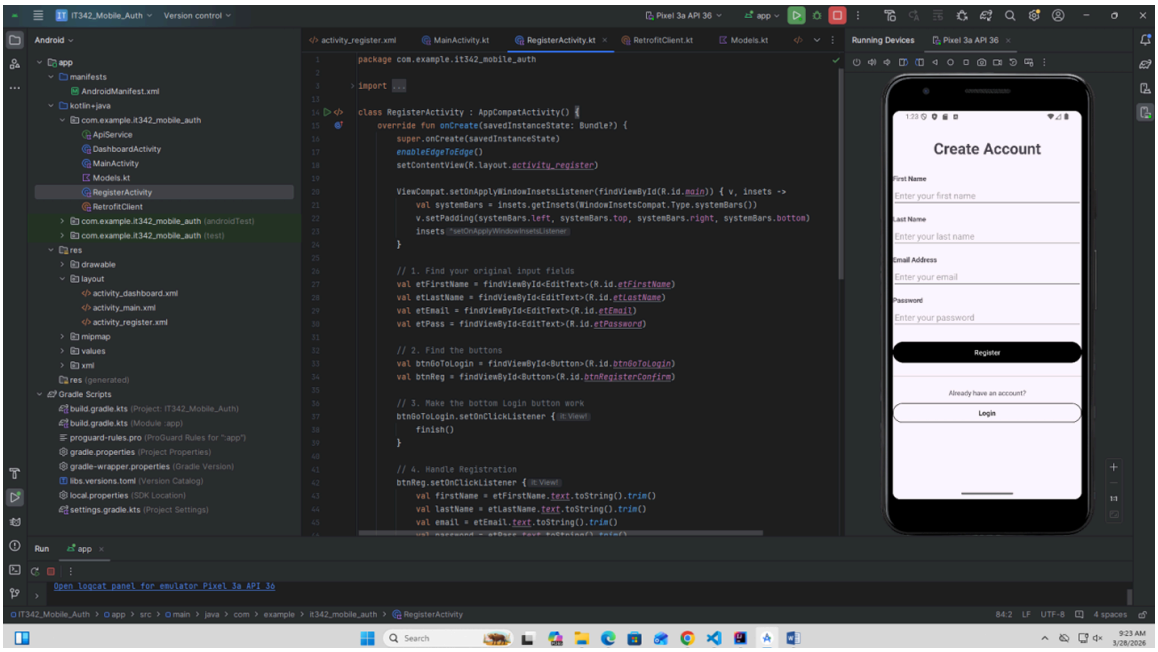
7. Web Application Screenshots

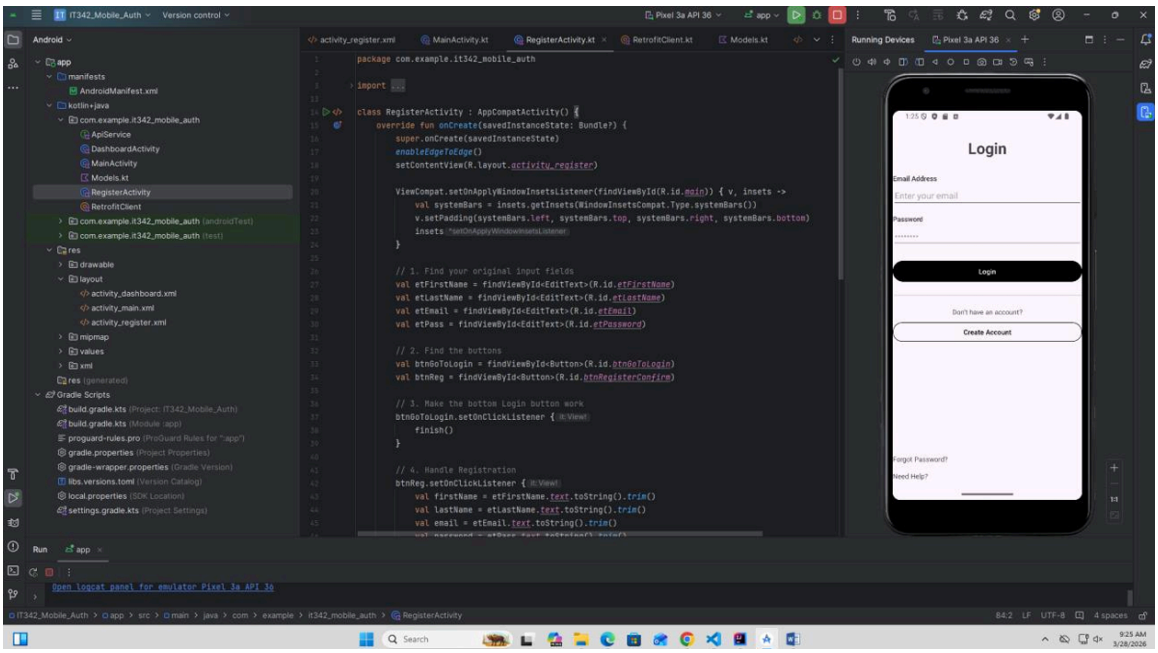


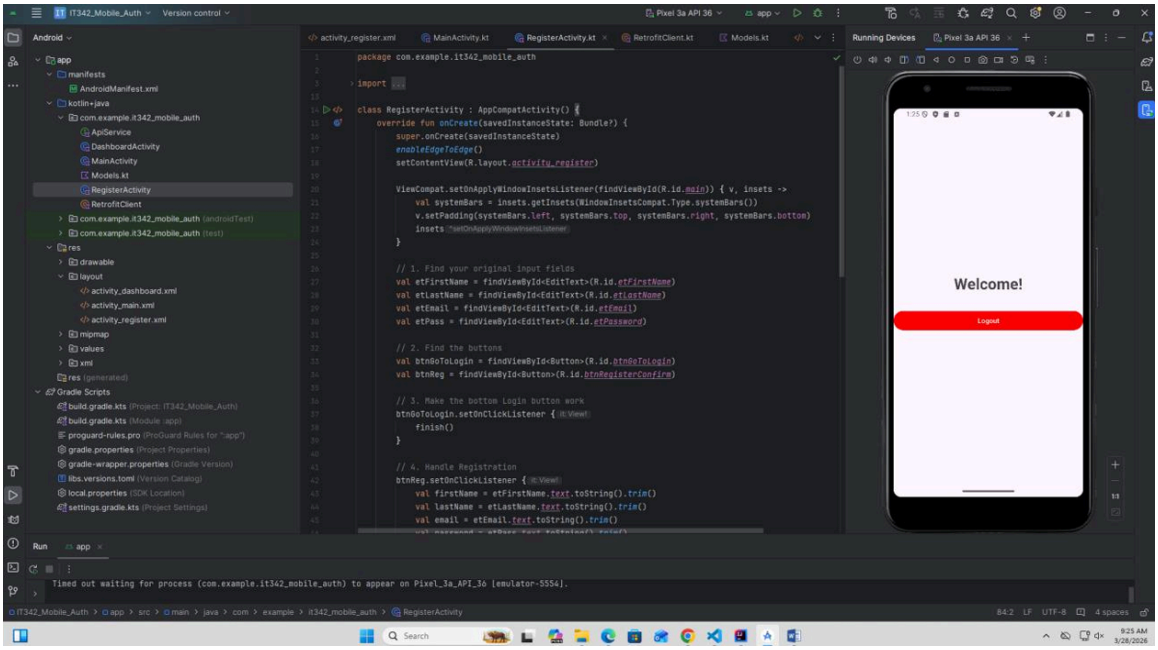



web
localhost:5173/home
RentEasy
Dashboard
Rentals
Inventory
Customers
Settings
Sign Out
Dashboard
Save password
josephnuevas@g...
Edit
No need to remember your passwords anymore
Got it
Not now
JD
Welcome back, John!
Here's what's happening with your rentals today.
+15%
Total Revenue
$12,450
+5
Active Rentals
45
-2
Pending Returns
12
+12%
New Customers
28
Recent Rentals
View All
ITEM NAME
CUSTOMER
STATUS
DATE
AMOUNT
Canon EOS R5
Sarah M.
Active
Oct 24, 2024
$120.00
DJI Mavic 3
Mike Ross
Pending
Oct 23, 2024
$85.00
Sony A7III Kit
Jessica P.
Returned
Oct 21, 2024
$200.00
Godox Light Set
Harvey S.
Active
Oct 20, 2024
$60.00
Search
10:17 AM
3/7/2026

8. Mobile Application Screenshots






IT342_Mobile_Auth
Version control
Pixel 3a API 36
app
Android
app
manifests
AndroidManifest.xml
kotlin+java
com.example.it342_mobile_auth
ApiService
DashboardActivity
MainActivity
Models.kt
RegisterActivity
RetrofitClient
com.example.it342_mobile_auth (androidTest)
com.example.it342_mobile_auth (test)
res
drawable
layout
activity_dashboard.xml
activity_main.xml
activity_register.xml
mipmap
values
xml
res (generated)
Gradle Scripts
build.gradle.kts (Project: IT342_Mobile_Auth)
build.gradle.kts (Module :app)
proguard-rules.pro (ProGuard Rules for ":app")
gradle.properties (Project Properties)
gradle-wrapper.properties (Gradle Version)
libs.versions.toml (Version Catalog)
local.properties (SDK Location)
settings.gradle.kts (Project Settings)
activity_register.xml
MainActivity.kt
RegisterActivity.kt
RetrofitClient.kt
Models.kt
package com.example.it342_mobile_auth
import ...
class RegisterActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
enableEdgeToEdge()
setContentView(R.layout.activity_register)
ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
insets
}
// 1. Find your original input fields
val etFirstName = findViewById<EditText>(R.id.etFirstName)
val etLastName = findViewById<EditText>(R.id.etLastName)
val etEmail = findViewById<EditText>(R.id.etEmail)
val etPass = findViewById<EditText>(R.id.etPassword)
// 2. Find the buttons
val btnGoToLogin = findViewById<Button>(R.id.btnGoToLogin)
val btnReg = findViewById<Button>(R.id.btnRegisterConfirm)
// 3. Make the bottom Login button work
btnGoToLogin.setOnClickListener {
finish()
}
// 4. Handle Registration
btnReg.setOnClickListener {
val firstName = etFirstName.text.toString().trim()
val lastName = etLastName.text.toString().trim()
val email = etEmail.text.toString().trim()
Running Devices
Pixel 3a API 36
Welcome!
Logout
Run
app
Timed out waiting for process (com.example.it342_mobile_auth) to appear on Pixel_3a_API_36 [emulator-5554].
IT342_Mobile_Auth > app > src > main > java > com > example > it342_mobile_auth > RegisterActivity
84:2
LF
UTF-8
4 spaces
Search
9:25 AM
3/28/2026